



Desafio prático - Desenvolvedor Full Stack Java e Angular

Este é um roteiro das atividades do desafio para a vaga de Desenvolvedor Full Stack Java e Angular. O desafio consiste em desenvolver um sistema web para cadastro, atualização e visualização de usuários com suas biometrias faciais (foto do rosto), além de permitir a verificação (1:1) e a identificação (1:n) de usuários a partir de uma foto.

Este desafio aborda 5 pontos principais, sendo eles: criação de um banco de dados relacional, uma API REST, um front-end, processamento de dados em paralelo e o uso de biometria facial para verificação e identificação de usuários.

O objetivo é avaliar o candidato no uso das tecnologias indicadas e das boas práticas de programação considerando o nível esperado (júnior, pleno ou sênior). A implementação é livre, use sua criatividade!

Observações:

- É fácil encontrar código de exemplo pronto para este propósito na Internet;
- Você pode se basear nestes códigos, mas espera-se que o sistema final entregue seja robusto e estável, o que normalmente não é o caso de códigos de exemplo;
- Utilize seu conhecimento e os métodos adequados para garantir/provar esta robustez e estabilidade;
- Comentários em código e tratamentos de erro são bem-vindos;
- Utilize Java 8 ou versão mais atual;
- Utilizar o framework SpringBoot para a API REST e integração com o banco de dados;
- Utilizar o banco de dados Postgres 12 ou superior;
- Utilizar o framework Angular ou superior para o front-end;
- É desejável, mas não obrigatório, o uso do Maven para gestão de dependências;
- Caso não consiga concluir o desafio, entregue-o mesmo assim.

Entregáveis

- A entrega do desafio pode ser realizada preferencialmente utilizando um repositório GIT de sua preferência e fornecendo (opcionalmente) uma imagem docker para

execução do sistema;

Descrição do Sistema - CRUD de biometrias faciais

O sistema a ser desenvolvido baseia-se em um CRUD - acrônimo para Create (criar), Read (ler), Update (atualizar) e Delete (apagar)) - de biometrias faciais (foto do rosto) de um usuário.

Nesse sistema deve ser possível por meio do front-end Angular, cadastrar, atualizar e remover usuários e suas fotos. É importante destacar que um usuário **precisa** ter uma foto vinculada.

Deve ser desenvolvida uma API REST em Java SpringBoot para acessar e modificar os dados, assim como utilizar o Java para realizar a conexão com o banco de dados Postgres.

Além do CRUD, o sistema deve oferecer dois recursos de biometria facial descritos na seção Requisitos de Biometria Facial: verificação (match 1:1) e identificação (match 1:n).

Descrição dos requisitos

- **Back-end em Java:**
 - Utilize o Java SpringBoot para criar a API necessária para a integração do front-end com o back-end, assim como para realizar a integração do banco de dados e controle de concorrência.
- **Integração com Banco de Dados Postgres:**
 - Utilize o Postgres para armazenar os dados dos usuários, incluindo CPFs, nomes dos usuários e fotos.
- **Front-end em Angular:**
 - Desenvolva uma interface de usuário utilizando Angular para interagir com o sistema. Não é necessário dedicar muito tempo para o design da interface, ele não será avaliado, apenas a funcionalidade e simplicidade;
 - Crie telas para cadastro, atualização, listagem, exibição e exclusão de usuários;
 - Crie telas para verificação (1:1) e identificação (1:n) de biometrias faciais (detalhadas adiante)
- **Operações:**
 - **Cadastro:** Permita o envio de uma foto, nome e CPF do usuário. Persista esses dados no banco de dados Postgres. Permita o cadastro/atualização de múltiplos usuários ao mesmo tempo. Use threads separadas para cada inserção e trate a concorrência de cadastro e atualizações de fotos corretamente;
 - **Atualização:** Permita a atualização do nome e da foto de um usuário;

- **Persistência de Dados:** Garanta que o cadastro só seja efetivado quando todos os dados estiverem salvos corretamente no banco;
- **Tela de Listagem:** Crie uma tela para listar usuários;
- **Exibição:** Desenvolva uma tela de detalhes exibindo: CPF, nome e foto do usuário.
- **Verificação facial (1:1):** Desenvolva uma tela em que o usuário fornece um identificador de usuário e uma foto, e o sistema retorna se há ou não correspondência, entre a foto fornecida e a foto cadastrada no sistema para aquele usuário;
- **Identificação facial (1:n):** Desenvolva uma tela em que o usuário fornece apenas uma foto e o sistema retorna o usuário correspondente (ou indica que não houve identificação).

Requisitos de Biometria Facial (Verificação e Identificação)

Além do CRUD, o sistema deve ser capaz de comparar rostos. Duas operações distintas são exigidas, e é fundamental que o candidato compreenda a diferença entre elas:

- **Verificação (1:1):** A foto enviada é comparada com a foto de um único usuário específico já cadastrado.
- **Identificação (1:n):** A foto enviada é comparada com as fotos de todos os usuários cadastrados, a fim de descobrir a qual usuário o rosto pertence (ou concluir que não há correspondência).

Em ambos os casos, o fluxo conceitual é o mesmo e envolve quatro etapas: (1) detectar o rosto na imagem enviada; (2) extrair uma representação numérica do rosto (comumente chamada de template facial); (3) calcular uma medida de similaridade entre os rostos; e (4) tomar uma decisão com base em um limiar (threshold) previamente definido. Acima do limiar, considera-se que os rostos pertencem à mesma pessoa; abaixo dele, considera-se que são pessoas diferentes.

Observação: Para essas funcionalidades, o candidato pode escolher livremente as bibliotecas de processamento de imagem ou as APIs de visão computacional de sua preferência. O foco da avaliação não é treinar um modelo de reconhecimento facial do zero, e sim integrar uma solução existente de forma correta, tratando adequadamente o score de similaridade, o limiar de decisão e os casos de erro.

Verificação de biometria facial (1:1)

Compara uma foto fornecida com a foto armazenada de um usuário específico para confirmar (ou não) a identidade.

- **Entrada:** um identificador do usuário (CPF) e uma foto a ser verificada.
- **Processamento:** o back-end deve localizar o usuário informado, comparar a foto recebida com a foto cadastrada desse usuário e calcular um score de similaridade.
- **Saída:** uma resposta indicando se houve correspondência (match verdadeiro/falso) e o score de similaridade obtido.
- **Front-end:** uma tela onde o usuário fornece o identificador do usuário, envia uma foto e visualiza o resultado (compatível ou não compatível), além do score retornado.

Identificação de biometria facial (1:n)

Compara uma foto fornecida com a base de todos os usuários cadastrados para identificar a qual usuário o rosto pertence.

- **Entrada:** apenas uma foto a ser identificada (sem informar previamente quem é a pessoa).
- **Processamento:** o back-end deve comparar a foto recebida com as fotos de todos os usuários cadastrados e selecionar o melhor candidato (maior score de similaridade) que esteja acima do limiar definido.
- **Saída:** uma resposta indicando se algum usuário foi encontrado e, em caso positivo, o usuário correspondente com seu respectivo score de similaridade. Caso nenhum resultado ultrapasse o limiar, a resposta deve indicar claramente que não houve identificação.
- **Front-end:** uma tela onde o usuário envia apenas uma foto e visualiza o resultado da identificação (usuário identificado, ou mensagem de "não identificado"), incluindo o score.

Pontos de atenção (válidos para as duas operações)

- **Limiar de decisão:** deixe claro no código qual limiar é usado para decidir se duas fotos pertencem à mesma pessoa. É um diferencial torná-lo configurável (por exemplo, via arquivo de configuração ou variável de ambiente) em vez de "fixo no código".
- **Tratamento de casos de erro:** trate de forma adequada situações como imagem sem rosto detectável, imagem com mais de um rosto, formato de arquivo inválido, arquivo corrompido ou usuário inexistente (no caso da verificação). A API deve retornar mensagens e códigos de status HTTP apropriados.
- **Desempenho na identificação (1:n):** a identificação tende a ficar mais custosa conforme a base de usuários cresce, pois envolve múltiplas comparações. Pense em como tornar essa operação um pouco mais eficiente — por exemplo, processando as comparações em paralelo e/ou pré-calculando e armazenando os templates dos

rostos no momento do cadastro. Não é necessário utilizar estratégias complexas de 1:n, uma comparação linear já é suficiente.

Validação da implementação

Para auxiliar nos testes da implementação, recomendamos o uso do software Postman, que possui uma interface simples para configurar uma requisição. Você pode fornecer junto a API Postman utilizada.

Os pontos a serem verificados para que o desafio possa ser considerado concluído, são os seguintes:

- Start do servidor sem erros;
- Endpoint conseguem receber dados e retornar uma resposta;
- Tratamento de erros;
- Tratamento de concorrência;
- Front-end funcional;
- Armazenamento dos dados no Banco de Dados;
- Verificação facial funcional
- Identificação facial funcional

Links úteis

- SpringBoot
 - o <https://spring.io/>
- OpenCV (visão computacional):
 - o <https://opencv.org/>
- JavaCV (binding de OpenCV para Java):
 - o <https://github.com/bytedeco/javacv>
- Deep Java Library - DJL (modelos de IA em Java, inclui reconhecimento facial):
 - o <https://djl.ai/>
- Docker
 - o <https://docs.docker.com/>

- Tutorial criar API Rest com SpringBoot
 - o <https://spring.io/guides/gs/rest-service/>